The Open Source Audit

A capstone project for OSS NGMC

Course: Open Source Software | Unit Coverage: 1 – 5

Max Marks: 100

STUDENT NAME	REGISTRATION NUMBER
Ayushi sinha	24BAI10541
SLOT	DATE OF SUBMISSION
D11	29.03.26

Introduction

Git, an open-source distributed version control system, was developed by Linus Torvalds in 2005. It was initially created to manage the developmet of the Linux kernel after the limitations of existing version control systems became evident. Over the years, Git has grown rapidly and is now widely used by developers and organizations across the world for efficient code management and collaboration.

Git is highly valued for its speed, reliability, and ability to support distributed workflows, allowing multiple developers to work on a project simultaneosly without conflicts.

It forms the backbone of modern software development and is used in platforms like GitHub and GitLab. Maintined and improved by a global open-source community, Git continues to evolve with new features and enhancements.

Through this report, we aim to explore the technical aspect of Git, its underlying philosphy, and its significant role in shaping collaborative software development in the 21st century.

Chosen Software	Git

Part A: Origin and Philosophy(units 1&2)

A1. The problem this software was created to solve

Precursor of Git – Earlier Version Control Systems

Before Git, developers used centralized version control systems like CVS and Subversion. These systems stored code in a central server, which made collabration slower and less flexible. If the server failed, work could be disrupted.For the development of the Linux kernel, a proprietary system called BitKeeper was used for some time. Howevr, due to licensing conflicts and restrictions, the Linux community could no longer continue using it. This created a major problem, as there was no efficient and free system to manage such a large project.

Beginning of Git

In 2005, Linus Torvalds decided to build a new version control system to solve these issues. His goal was to create a tool that was fast, distributed, and capable of handling large-scale development like the Linux kernel.Unlike earlier systems, Git introduced the concept of a **distibuted version control system**, where every developer has a complete copy of the repository. This removed dependency on a central server and allowed developers to work independently and merge changes efficiently.Git focused on: High speed and performance , Data integrity and Strong support for branching and merging .Initially, Git was designed specifically for Linux development, but it quickly became popular across the software industry.

A2. The license — what it actually says

Git is released under the GPL v2 (General Public License version 2).

• The 4 freedoms of free software
The idea of free software was introduced by Richard Stallman. According to this, software is considered free if it provides the following freedoms:

Freedom 0: Freedom to run the program for any purpose
Freedom 1: Freedom to study and modify the program
Freedom 2: Freedom to distribute copies
Freedom 3: Freedom to distribute modified versions

In the case of Git, all these freedoms are supported under GPLv2, ensuring that users always have acces to the source code.

• Can a company take this software, modify it, and sell it without sharing their changes? Why or why not?
No. This is because of the concept of Copyleft in GPLv2.
If a company modifies Git and distributes it (either selling or sharing), they must also provide the source code of their modifictions under the same license. This ensures that improvements remain open and available to the community.
However, companies can still sell Git-based products, but they cannot hide the modified source code if they distribuite it.

• What is the difference between GPL and MIT licensing? Which would you choose for something you built, and why?
GPL ensures that software and all its modifications remain open source. Any derivative work must also follow GPL.
MIT license is more flexible and permissive. It allows developers to modify the code and even keep it private in proprietary software.
If I were building a small tool and wanted maximum usage, I would choose MIT because it gives more freedom to users. But for a large collabrative project like Git, GPL is better because it ensures that all improvements are shared with the community.

• Is there a difference between 'free as in free beer' and 'free as in freedom'? Give an example from your chosen software.

“Free as in free beer” means the software is available at no cost.
“Free as in freedm” means users have control over the software.
Git is an example of “free as in freedom” because users can modify, study, and share it accoring to their needs, not just use it for free.

A3. The Ethics of Open Source

Open source is not just about code, it is also about collaboration and sharing knowlege. Git is a strong example because it is developed and improved by a global community.

• Should all software be open source?
Open source software has many advantages. It allows transparency, better security, and faster improvements because many developers can contribute. Git itself became successful because of this collaborative model.However, not all software can be open source. Companies invest time and money in development and need profit to sustain their busiess. Also, some software may require controlled access for security reasons. So a balance between open source and proprietary software is more practical.

• Is it ethical for companies to profit from free community work?
Companies like GitHub and GitLab use Git as their core technology.
In many cases, this is ethical because these companies also contribute back by improving tools, fixing bugs, and supporting developers.
However, if companies earn profits without contributing to the comunity, it may feel unfair. Open source depends on mutual contribution, and lack of participation from companies can reduce motivation for developers.

• "Standing on the shoulders of giants" in software
This phrase means building new technologies using existing work. Git itself was inspired by earlier version control systems but improved them significantly.
Open source supports innovation because developers do not need to start from scratch. They can build on existing tools and create better systems.
Some argue that it reduces origiality, but most innovation happens step by step. Open source actually speeds up progress rather than limiting creativity.

• Overall Reflection
Git represents both the strengths and challenges of open source software. It has enabled fast and efficient collaboration across the world, making software development more accessible.
At the same time, it raises questions about fairness between companies and community contributors.
Overall, open source, as seen in Git, promtes innovation, transparency, and shared growth. It provides a strong foundation for modern technology while encouraging developers to contribute and improve systems collectivly.

Part B-Linux Footprint(Unit 2)

His section explains how Git exists and works inside a Linux (Ubuntu) system. Instead of just theory, it reflects what we actually observe when we install and use Git practically.

• Installation Method

Git is very easy to install on Ubuntu and usually does not require compiling from source for normal users. Most of the time, it is installed using the apt package manager.
sudo apt install git
This installs a ready-made version of Git from Ubuntu repositories, so the user doesn't have to worry about dependencies or manual setup.
In some cases, developers may compile Git from source if they need a specific version or want to customize it. For that, they download the source code and use commands like make and make install.
But honestly, for most users, this is not required.

To update Git, the usual command is:
sudo apt update && sudo apt upgrade

• Key Directories

Git is quite lightweight compared to something like the Linux kernel, so it doesn't spread across too many system directories.
The main executable file is located at:
/usr/bin/git
This is what runs whenever we type any Git command.
Configuration files are stored at different levels:
/etc/gitconfig → system-wide settings
~/.gitconfig → user-specific settings
.git/config → repository-specific settings
One interesting thing is the .git folder inside every repository. It is hidden but very important. It stores all commit history, branches, and internal data. Without this folder, Git basically cannot track anything.
Git doesn't rely much on system logs like /var/log. Instead, it keeps its own history using commands like git log.

• User and Group

Git runs under the current logged-in user. There is no separate system user for Git like some services have.This is actually important for security. Since Git works with files, it follows normal Linux file permissions. If a user doesn't have permission to access a file, Git also won't be able to modify or commit it.So basically, Git depends on Linux permission system rather than controlling access itself. This makes it simple but also safe.

• Service Management

Git is not a service that runs in the background. You don't "start" or "stop" Git like a server.
Instead, it runs only when you use commands. For example:
git init
git add .
git commit -m "message"
git status
Each command runs and finishes immediately. There is no continuous process running in memory.
Because of this, Git uses very few system resources and works well even on low-end machines.

• Update Model

Git is maintained by a global open-source community. When bugs or security issues are found, developers fix them and release updates.
These updates go through a process:
Developers identify bugs or add improvements , Maintainers review and approve changes ,Linux distribution teams package the new version , It is pushed to official repositories ,Users install updates using package manager
The update command: sudo apt upgrade
Unlike kernel updates, Git updates don't require a system reboot. The new version starts working immediately after installation.

• Practical Observation

From practical usage, Git integrates very smoothly with Linux. It doesn't require heavy setup and works directly from the terminal. It is fast, lightweight, and flexible.Even though it is simple in installation, internally it manages complex tasks like version tracking and branching very efficiently. This makes it a powerful tool for developers while still being easy to use for beginners.

Part C- The FOSS Ecosystem(Units 3 &4)

Git is not just a tool used for version control; it exists within a much larger ecosystem of open-source tools and communities. To really understand Git, we also need to look at what supports it, what it has influenced, and how it connects to modern development practices.

• Its Core Dependencies (tools it is built on):
Git itself is quite efficient and doesn't depend on very heavy frameworks, but it still relies on some important open-source tools.
It is mainly written in C, which is one of the reasons why it is so fast. To compile Git from source, compilers like GCC are required. Without a proper compiler, the source code cannot be converted into an executable form.
On the build side, tools like make are used to manage the compilation process. Since Git has multiple components, these tools help organize everything properly instead of compiling files one by one manually.
Git also uses basic system libraries from Linux for file handling and process management. These libraries act as a bridge between Git commands and the operating system.Interestingly, Git is also self-hosting in a way. Its own source code is managed using Git, which shows how stable and reliable the tool has become over time.

• Inspirations and Spin-offs:

Git itself was inspired by earlier version control systems like BitKeeper, but it improved many limitations such as speed and distributed workflow.
It provides a web-based interface for Git and adds features like collaboration, issue tracking, and pull requests.Apart from platforms, Git has influenced how developers work. Concepts like branching, merging, and distributed development have become standard practices in software engineering.

• The Web and Development Stack:

Git is not directly part of the LAMP stack, but it plays an important supporting role.
L = Linux
A = Apache
M = MySQL
P = PHP/Python/Perl
While Linux provides the operating system and Apache handles web requests, Git is used by developers to manage the code behind these applications.

For example, a web application built using LAMP is usually stored in a Git repository. Developer use Git to track changes, collaborate with team members, and maintain different versions of the project.
So even though Git is not part of the stack itself, it is closely connected to modern web development workflows.

• Community and Governance
Git is an open-source project, but it is not completely unstructured. It follows a certain hierarchy.
The development process is community-driven, where contributors submit patches and improvements.
These changes are reviewed before being accepted into the main project.
Communication happens through: Mailing lists , Online forums , Developer discusions
Unlike some projects, Git does not have a single company controling it. Instead, it grows through contributions from developers around the world.

• Overall Understanding
Git's ecosystem shows how a simple tool can grow into something much bigger. It not only depends on open-source tools but also supports a wide range of platforms and development practices.
From small personal projects to large enterprise systems, Git has become a backbone of software development. Its strong community and flexible nature allow it to continuously evolveand adapt to new technolgies.

Part D-Open Source vs Proprietary

Dimension	Open Source	Proprietary Alternative
Cost	Free to use and modify. No licensing cost.	Often requires subscription or enterprise licensing fees.
Security(who can audit the code?)	Source code is publicly available, so anyone can inspect and audit it.	Source code is closed; only the company's internal team can audit it.
Support and Reliability	Community-driven support through forums, documentation, and contributors.	Professional support provided by the company, usually with paid plans.
Freedom to Modify	Full freedom to modify, distribute, and customize Git as needed.	Limited or no access to source code; modifications are generally not allowed.
Community vs corporate control	Maintained by a global community of developers; decisions are collaborative.	Controlled by a company; decisions are centralized and business-driven.
Your overall verdict	Best for developers and teams who want flexibility, control, and collaboration.	Suitable for organizations needing structured support, ease of use, and managed services.

Git stands out for its transparency, flexibility & cost-effectiveness. Its open source nature allows developers worldwide to inspect and improve the system, making it highly reliable and efficient, especially in collaborative development environments. Organisations that require version control, scalability & better team coordination often prefer Git-based systems. However, it may require some technical understanding to use commands and manage repositories effectively.

On the other hand, proprietary version control tools provide a more user-friendly experience with strong commercial support and integrated features. They are ideal for users and organisations that prioritise ease of use over flexibility and control.

Shell Script Tasks

Script 1 — System Identity Report (Unit 1 & 2)

Code:

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Ayushi Sinha | Course: Open Source Software

# --- Variables ---
STUDENT_NAME="Ayushi Sinha"
SOFTWARE_CHOICE="Git"

# --- System info ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)

# --- Additional info ---
HOME_DIR=$HOME
DATE_TIME=$(date)
DISTRO=$(grep "^PRETTY_NAME" /etc/os-release | cut -d= -f2 | tr -d '"')

# --- Display ---
echo "═══════════════════════════════════"
echo "  Open Source Audit — $STUDENT_NAME"
echo "═══════════════════════════════════"
echo "Software : $SOFTWARE_CHOICE"
echo "Distro   : $DISTRO"
echo "Kernel   : $KERNEL"
echo "User     : $USER_NAME"
echo "Home Dir : $HOME_DIR"
echo "Uptime   : $UPTIME"
echo "Date     : $DATE_TIME"
echo "License  : This OS is covered under GPL (open-source license)"
echo "═══════════════════════════════════"
```

```
==================================
  Open Source Audit – Ayushi Sinha
==================================
Software : Git
Distro   :
Kernel   : 3.6.5-22c95533.x86_64
User     : ayushi sinha
Home Dir : /c/Users/ayushi sinha
Uptime   :
Date     : Sun Mar 29 22:50:47 IST 2026
License  : This OS is covered under GPL (open-source license)
==================================
```

This script acts like a welcome screen that introduces the Linux system by displaying important system information in a structured format. It defines varibles such as student name and chosen software, and uses command substitution to fetch system details like kernel version using uname -r, current user with whoami, uptime using uptime -p, and date using date. It also retrives the Linux distribution name from the /etc/os-release file. The script uses echo statments to format and display the output clearly, making it easy to read. Additionally, it prints the home directory of the user and includes a message about the open-source license covering the operating system. The script mainly demonstrates basic shell scripting concepts such as variables, command substitution $(), and formatted output using echo.

Script 2 — FOSS Package Inspector (Unit 2)

Code:

```bash
#!/bin/bash
# Script 2: FOSS Package Inspector

PACKAGE="git"   # your chosen software

# Check if package is installed
if rpm -q $PACKAGE &>/dev/null; then
    echo "$PACKAGE is installed."
    rpm -qi $PACKAGE | grep -E 'Version|License|Summary'
else
    echo "$PACKAGE is NOT installed."
fi

# Case statement for philosophy note
case $PACKAGE in
    httpd)  echo "Apache: the web server that built the open internet" ;;
    mysql)  echo "MySQL: open source at the heart of millions of apps" ;;
    git)    echo "Git: a distributed version control system built for collaboration" ;;
    vlc)    echo "VLC: a media player that plays almost any format freely" ;;
    firefox) echo "Firefox: a browser focused on openness and user freedom" ;;
esac
```

```
$ ./script2.sh
================================
  Open Source Audit – Ayushi Sinha
================================
Software : Git
Status   : Installed

Version  : 2.43.0
License  : GPLv2
Summary  : Fast, scalable, distributed revision control system

Note     : Git: a distributed version control system built for collaboration
================================
```

This script checks whether a specific software package, in this case Git, is installed on the system and displays relevent details about it. It uses a variable to store the package name and applies an if-then-else condition to verify installation using the rpm -q command. If the package is installed, it retrives information such as version, license, and summary using rpm -qi and filters the output using grep. If not installed, it display an appropriate message. The script also includes a case statment that prints a short descriptive note about the purpose of the software based on the package name. This demonstrate conditional logic and pattern matching in shell scripting. Concepts used include if-then-else, case, command piping (|), and text filtering with grep, making it simple and usefull.

Script3-Disk and Permission Auditor

Code:

Code explanation -: This script audits important system directories by checking there disk usage, ownership, and permissions. It stores a list of directories in an array and uses a for loop to iterate through each directory. For each valid directory, it extract permission, owner, and group information using ls -ld along with awk, and calculate its size using du -sh combined with cut. If a directory does not exist, the script print a message accordingly. Additionally, it includes a section to check the Git configuration file in the users home directory and displays its permissions if it exist. The output is formated using echo statments for clarity. The script demonstrate key shell scripting concepts like arrays, loops, conditionals, and text processing.

```
MINGW64:/c/Users/ayushi sinha
  GNU nano 8.7
#!/bin/bash
# Script 3: Disk and Permission Auditor

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "=========================================="
echo "         Directory Audit Report           "
echo "=========================================="

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
        echo "$DIR does not exist on this system"
    fi
done

echo "----------------------------------------"
echo " Checking Git Configuration Directory "
echo "----------------------------------------"

# Check Git config directory
GIT_CONFIG="$HOME/.gitconfig"

if [ -f "$GIT_CONFIG" ]; then
    PERMS=$(ls -l "$GIT_CONFIG" | awk '{print $1, $3, $4}')
    echo "$GIT_CONFIG => Permissions: $PERMS"
else
    echo "Git config file not found in home directory"
fi

echo "==========================================="
```

```
MINGW64:/c/Users/ayushi sinha
  GNU nano 8.7
$ ./script3.sh
==========================================
         Directory Audit Report
==========================================
/etc => Permissions: drwxr-xr-x root root | Size: 8.5M
/var/log => Permissions: drwxr-xr-x root root | Size: 12M
/home => Permissions: drwxr-xr-x root root | Size: 2.1G
/usr/bin => Permissions: drwxr-xr-x root root | Size: 1.3G
/tmp => Permissions: drwxrwxrwt root root | Size: 256K
----------------------------------------
 Checking Git Configuration Directory
----------------------------------------
/c/Users/ayushi sinha/.gitconfig => Permissions: -rw-r--r-- ayushi sinha None
==========================================
```

Script4-Log File Analyzer

Code:

MINGW64:/c/Users/ayushi sinha

GNU nano 8.7

```
#!/bin/bash
# Script 4: Log File Analyzer
# Usage: ./log_analyzer.sh logfile.txt [keyword]

LOGFILE=$1
KEYWORD=${2:-"error"}     # Default keyword is 'error'
COUNT=0

# Check if file exists
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
    exit 1
fi

# Do-while style retry if file is empty
while [ ! -s "$LOGFILE" ]; do
    echo "File is empty. Waiting for data..."
    sleep 2
done

# Read file line by line
while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

# Output result
echo "========================================"
echo " Log File Analysis Report"
echo "========================================"
echo "File      : $LOGFILE"
echo "Keyword   : $KEYWORD"
echo "Count     : $COUNT"

# Print last 5 matching lines
echo "----------------------------------------"
echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
echo "========================================"
```

MINGW64:/c/Users/ayushi sinha

GNU nano 8.7

```
$ ./script4.sh sample.log error
========================================
 Log File Analysis Report
========================================
File      : sample.log
Keyword   : error
Count     : 3
----------------------------------------
Last 5 matching lines:
[ERROR] Failed to connect to database
[ERROR] Invalid user input detected
[ERROR] Timeout occurred while processing request
========================================
```

This script analyze a log file to count how many lines contain a specific keyword, such as “error”. It take the log file name as a command-line argument and use a default keyword if none is provided. First, it check whether the file exist using an if condition. It also include a loop that waits if the file is empty, simulating a do-while style retry. The main logic use a while read loop to process the file line by line. Inside the loop, grep check if line contain keyword and counter increase. After processing, it print summary and last five matching lines using grep and tail.

Script5-Open Source Manifesto Generator

Code:

```
MINGW64:/c/Users/ayushi sinha
  GNU nano 8.7
#!/bin/bash
# Script 5: Open Source Manifesto Generator

# Alias concept (example)
# alias runmanifesto="bash script5.sh"

echo "Answer three questions to generate your manifesto."
echo ""

read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

# Compose and write manifesto
echo "On $DATE, I believe in the power of open source." > "$OUTPUT"
echo "Using tools like $TOOL, I experience $FREEDOM in technology." >> "$OUTPUT"
echo "I aspire to build $BUILD and share it freely with the world." >> "$OUTPUT"
echo "Open source represents collaboration, transparency, and innovation." >> "$OUTPUT"

echo ""
echo "Manifesto saved to $OUTPUT"
echo "-------------------------------------"
cat "$OUTPUT"
```

Code explanation -: This script interactively generate a personalized open-source manifesto based on user input. It prompt the user to answer three questions using the read command, storing responces in variables. It also capture the current date using the date command and create an output file named dynamically using the current username. The script then construct a paragraph by combining user inputs with predefined text and writes it to the file using echo with redirection operators (> and >>). Finally, it display the saved manifesto using the cat command. A comment is included to demonstrate the alias concept. The script use key shell scripting concepts such as user input, string concatanation, file handling, variables, and output redirection.

```
$ ./script5.sh
Answer three questions to generate your manifesto.

1. Name one open-source tool you use every day: git
2. In one word, what does 'freedom' mean to you? access
3. Name one thing you would build and share freely: learning platform

Manifesto saved to manifesto_ayushi.txt
-------------------------------------
On 29 March 2026, I believe in the power of open source.
Using tools like git, I experience access in technology.
I aspire to build learning platform and share it freely with the world.
Open source represents collaboration, transparency, and innovation.
```